



Qualcomm Technologies, Inc.

Resource Power Manager

API Reference

LM80-P0436-74 Rev. A

February 2, 2018

Qualcomm is a trademark of Qualcomm Incorporated, registered in the United States and other countries. Other product and brand names may be trademarks or registered trademarks of their respective owners.

This technical data may be subject to U.S. and international export, re-export, or transfer ("export") laws. Diversion contrary to U.S. and international law is strictly prohibited.

Qualcomm Technologies, Inc.
5775 Morehouse Drive
San Diego, CA 92121
U.S.A.

Revision history

Revision	Date	Description
A	February 2018	Initial release

Contents

1 Introduction.....	5
1.1 Purpose.....	5
1.2 Conventions	5
1.3 Technical assistance.....	5
2 Functional overview	6
3 RMP APIs.....	8
3.1 Data structure documentation	8
3.1.1 struct regulator_dev	8
3.1.2 struct rpm_regulator.....	9
3.1.3 struct msm_rpm_request.....	10
3.1.4 struct msm_rmp_kvp	10
3.1.5 struct platform device	10
3.2 Function documentation	11
3.2.1 static int msm_rpm_dev_probe(struct platform_device *pdev)	11
3.2.2 static int msm_rpm_send_data(struct msm_rpm_request *cdata,int msg_type, bool noirq).....	11
3.2.3 static int msm_rpm_send_smd_buffer(char *buf, uint32_t size, bool noirq) ..	11
3.2.4 int msm_rpm_send_message(enum msm_rpm_set set, uint32_t rsc_type,uint32_t rsc_id, struct msm_rpm_kvp *kvp, int nelems)	12
3.2.5 int msm_rpm_send_request(struct msm_rpm_request *handle)	12
3.2.6 void msm_rpm_free_request(struct msm_rpm_request *handle);	13
3.2.7 int msm_rpm_wait_for_ack(uint32_t msg_id)	13
3.2.8 static int rpm_vreg_send_request(struct rpm_regulator *regulator, u32 set) ..	13
3.2.9 static int rpm_vreg_set_voltage(struct regulator_dev *rdev, int min_uV,int max_uV, unsigned *selector)	14
3.2.10 static int rpm_vreg_get_voltage(struct regulator_dev *rdev).....	14
3.2.11 static int rpm_clk_prepare(struct clk *clk)	14
3.2.12 static void rpm_clk_unprepare(struct clk *clk)	15
3.2.13 static int rpm_clk_set_rate(struct clk *clk, unsigned long rate)	15
3.2.14 static unsigned long rpm_clk_get_rate(struct clk *clk)	15
A Source Code References.....	16
B References.....	17
B.1 Acronyms and terms	17

Figures

Figure 2-1 RPM functional block diagram 6

1 Introduction

1.1 Purpose

This API reference provides an interface to the Resource Power Manager (RPM). This document provides the public interfaces necessary to use the features provided by the RPM API.

1.2 Conventions

Function declarations, function names, type declarations, attributes, and code samples appear in a different font, for example, `#include`.

Code variables appear in angle brackets, for example, `<number>`.

1.3 Technical assistance

For assistance or clarification on information in this document, submit a case to Qualcomm Technologies, Inc. (QTI) at <https://createpoint.qti.qualcomm.com/>.

If you do not have access to the CDMATech Support website, register for access or send email to support.cdmatech@qti.qualcomm.com.

2 Functional overview

Figure 2-1 illustrates a functional block diagram of the RPM.

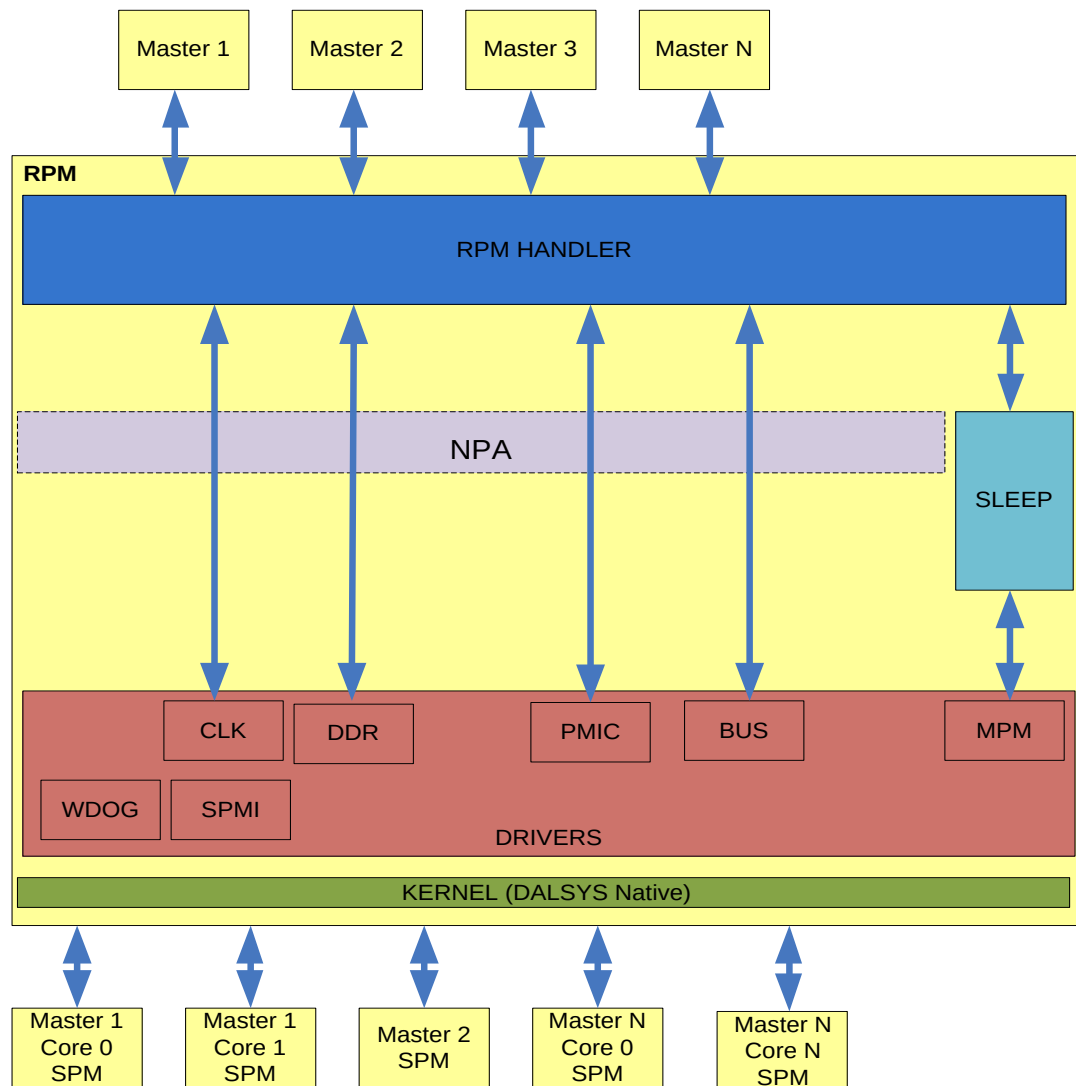


Figure 2-1 RPM functional block diagram

Modules in the RPM block diagram:

- Kernel: DALSYS Native-based lightweight kernel.
- RPM handler: RPM handler abstracts the RPM message protocol away from other software.
- Drivers: Drivers for each of the resources supported by the RPM register with the RPM handler to request notification when requests are received for the resource that the driver controls.
- Node power architecture (NPA): A driver may use the NPA to represent resources controlled by the driver.
- Clock driver: RPM clock driver directly handles the aggregating requests from each of the masters for any of the system-wide clock resources controlled by the RPM.
- Bus arbitration driver: RPM bus arbiter driver takes bus arbiter settings as requests from different masters in the system and aggregates them to represent the frequency-independent system settings.
- PMIC: RPM power management IC driver directly aggregates requests from each of the masters for any of the system-wide PMIC resources controlled by the RPM.
- Watchdog driver: Watchdog driver is a fail-safe mechanism for incorrect or stuck conditions.
- MPM driver: Modem power manager driver is used to program the MPM hardware block during system-wide sleep to monitor the wakeup-capable interrupts.

3 RMP APIs

The following are the APIs used in the RPM module:

- `static int msm_rpm_dev_probe(struct platform device *pdev)`
- `static int msm_rpm_send_data(struct msm_rpm_request *cdata, int msg_type, bool noirq)`
- `static int msm_rpm_send_smd_buffer(char *buf, uint32_t size, bool noirq)`
- `int msm_rpm_send_message(enum msm_rpm_set set, uint32_t rsc_type, uint32_t rsc_id, struct msm_rmp_kvp *kvp, int nelems)`
- `int msm_rpm_send_request(struct msm_rpm_request *handle)`
- `void msm_rpm_free_request(struct msm_rpm_request *handle)`
- `int msm_rpm_wait_for_ack(uint32_t msg_id)`
- `static int rpm_vreg_send_request(struct rpm_regulator *regulator, u32 set)`
- `static int rpm_vreg_set_voltage(struct regulator_dev *rdev, int min_uV, int max_uV, unsigned *selector)`
- `static int rpm_vreg_get_voltage(struct regulator_dev *rdev)`
- `static int rpm_clk_prepare(struct clk *clk)`
- `static void rpm_clk_unprepare(struct clk *clk)`
- `static int rpm_clk_set_rate(struct clk *clk, unsigned long rate)`
- `static unsigned long rpm_clk_get_rate(struct clk *clk)`

3.1 Data structure documentation

3.1.1 struct regulator_dev

Structure that contains regulator device information.

Data fields

Type	Field	Description
regulator_desc	*desc	Regulator descriptor.
int	exclusive	Exclusive variable.
u32	use_count	Use count.
u32	open_count	Open count.
u32	bypass_count	Bypass count.
list_head <redirect>	list	List of all regulators.

Type	Field	Description
list_head	consumer_list	Consumers we supply.
blocking_notifier_head <redirect>	notifier	Blocking notifier head.
mutex <redirect>	mutex	Consumer lock.
module	*owner	Module owner.
device <redirect>	dev	Device.
regulation_constraints <redirect>	*constraints	Regulation constraints.
regulator <redirect>	*supply	For the tree.
regmap <redirect>	*regmap	Regulator map.
delayed_work <redirect>	disable_work	Delayed work.
Int	deferred_disables	Deferred disables.
void	*reg_data	Regulator device data.
dentry <redirect>	*debugfs	Debug fs.
regulator_enable_gpio <redirect>	*ena_pin	Regulator enable GPIO.
unsigned int	ena_gpio_state:1	State of the GPIO.
proxy_consumer <redirect>	*proxy_consumer	Proxy consumer.
regulator	debug_consumer	Debug consumer.

3.1.2 struct rpm_regulator

Structure that contains RMP regulator information.

Data fields

Type	Field	Description
regulator_des	rdesc	Regulator descriptor.
regulator_dev	*rdev	Regulator device.
rpm_vreg	*rpm_vre	RPM regulator information.
list_head	list	List that links regulator data.
bool	set_active	Active set request.
bool	set_sleep	Sleep set request.
bool	always_send_voltage	Set if voltage is sent.
bool	always_send_current	Set if current is sent.
bool	use_pin_ctrl_for_enable	Set if pin control data is available in DTSI.
rpm_vreg_request	req	Request to the regulator at RPM.
int	system_load	Load in μ A present on regulator.
int	min_uV	Minimum μ V.
int	max_uV	Maximum μ V.
u3	pin_ctrl_mask[RPM_VREG_PIN_CTRL_STATE_COUNT]	RPM VREG pin control count.
rpm_regulator_param_index	voltage_index	Voltage index.
int	voltage_offset	Voltage offset.

3.1.3 struct msm_rpm_request

MSM RPM request structure.

Data fields

Type	Field	Description
rpm_request_header	req_hdr	Request handler header.
rpm_message_header	msg_hdr	Message header.
msm_rpm_kvp_data	*kvp	Data in key value pair (KVP) format.
uint32_t	num_elements	Number of elements in KVP data.
uint32_t	write_idx	Index ID of the SMD buffer.
uint8_t	*buf	SMD buffer.
uint8_t	numbytes	Number of bytes to be written.

3.1.4 struct msm_rmp_kvp

Message in KVP format.

Data fields

Type	Field	Description
uint32_t	key	4-byte string.
uint32_t	length	Number of bytes that follow.
uint32_t	*data	Pointer to the data.

3.1.5 struct platform_device

Platform device structure.

Data fields

Type	Field	Description
char	*name	Device name.
int	id	Device ID.
bool	id_auto	Automatically allocated device ID.
device	dev	Platform device data.
u32	num_resources	Number of resources associated with the platform device.
resource	*resource	Resource data.
platform_device_id	*id_entry	Driver name to force a match.
mfd_cell	mfd_cell	MFD cell pointer.
pdev_archdata	archdata	Architecture-specific additions.

3.2 Function documentation

3.2.1 static int msm_rpm_dev_probe(struct platform_device *pdev)

The APSS probes the RPM over SMD to communicate APSS to the RPM, and opens an RPM channel to communicate to the RPM.

Parameters

in	<i>pdev</i>	Platform device information.
----	-------------	------------------------------

Returns

If the probe is successful done, it returns TRUE, otherwise it returns the error EPROBE_DEFER.

Dependencies

Call `msm_rpm_dev_probe` during the MSM RPM driver initialization.

3.2.2 static int msm_rpm_send_data(struct msm_rpm_request *cdata, int msg_type, bool noirq)

Sends data over an SMD channel after checking for channel availability.

Parameters

in	<i>cdata</i>	Request to the RPM with a message header, KVP data, and the number of bytes to the SMD buffer.
in	<i>msg_type</i>	Message request type, such as clock, voltage, etc.
in	<i>noirq</i>	Do not enable interrupt.

Returns

If the API successfully writes to the SMD channel, it returns TRUE; otherwise it returns EBUSY.

Dependencies

Call `msm_rpm_dev_probe` before calling this function.

3.2.3 static int msm_rpm_send_smd_buffer(char *buf, uint32_t size, bool noirq)

Writes data to the open SMD channel.

Parameters

in	<i>buf</i>	Data to be written to the SMD channel.
in	<i>size</i>	Size of the buffer.
in	<i>noirq</i>	Do not enable interrupt.

Returns

If the API successfully writes to the SMD channel, it returns TRUE; otherwise it returns EBUSY.

Dependencies

Call `msm_rpm_dev_probe` before calling this function.

3.2.4 `int msm_rpm_send_message(enum msm_rpm_set set, uint32_t rsc_type, uint32_t rsc_id, struct msm_rpm_kvp *kvp, int nelems)`

Sends data or a message in KVP format.

Parameters

in	<i>set</i>	Set the active/sleep parameter for the resource.
in	<i>rsc_type</i>	Unsigned 32-bit integer that identifies the type of the resource.
in	<i>rsc_id</i>	Unsigned 32-bit integer that uniquely identifies a resource within a type.
in	<i>kvp</i>	Array of KVP data.
in	<i>nelems</i>	Number of KVP pairs associated with the message.

Returns

If an acknowledgement for the message is received from the RPM within the given time frame, the API returns 0 on success and an error on failure.

Dependencies

Call `msm_rpm_dev_probe` before calling this function.

3.2.5 `int msm_rpm_send_request(struct msm_rpm_request *handle)`

Sends the RPM messages using SMD. The function assigns a message ID before sending the data out to the RPM. The RPM hardware uses the message ID to acknowledge the messages.

Parameters

in	<i>handle</i>	Pointer to the <code>msm_rpm_request</code> for the resource being modified.
----	---------------	--

Returns

Returns a non-zero message ID on success and zero on a failed transaction.

The drivers use the message ID to wait for an acknowledgement from the RPM.

Dependencies

Call `msm_rpm_dev_probe` before calling this function.

3.2.6 void msm_rpm_free_request(struct msm_rpm_request *handle);

Clears the RPM request handle created with `msm_rpm_create_request`.

Parameters

in	<i>handle</i>	RPM resource handle to be cleared.
----	---------------	------------------------------------

Returns

None.

Dependencies

Call `msm_rpm_dev_probe` before calling this function.

3.2.7 int msm_rpm_wait_for_ack(uint32_t msg_id)

Defines how long to wait for the acknowledgement from the RPM that was sent from the APSS.

Parameters

in	<i>msg_id</i>	Return from <code>msm_rpm_send_request</code> .
----	---------------	---

Returns

If an acknowledgement is received by the APPS within the specified time, the API returns 0; otherwise, it returns an error number.

Dependencies

Call `msm_rpm_dev_probe` before calling this function.

3.2.8 static int rpm_vreg_send_request(struct rpm_regulator *regulator, u32 set)

Sends regulator data, such as the resource name (LDO or SMPS).

Parameters

in	<i>regulator</i>	Regulator-specific information.
in	<i>set</i>	Active or Sleep mode.

Returns

If an acknowledgement is received by the APPS within the specified time, the API returns TRUE; otherwise, it returns an error message.

Dependencies

Call `msm_rpm_dev_probe` before calling this function.

3.2.9 static int rpm_vreg_set_voltage(struct regulator_dev *rdev, int min_uV, int max_uV, unsigned *selector)

Aggregates the voltage and sends the aggregated voltage request to the RPM over an SMD channel.

Parameters

in	<i>rdev</i>	Voltage/current regulator class device; one for each regulator.
in	<i>min_uV</i>	Minimum microvolts.
in	<i>max_uV</i>	Maximum microvolts.
in	<i>selector</i>	Selector.

Returns

Returns the final set voltage value after aggregation.

Dependencies

Call `msm_rpm_dev_probe` before calling this function.

3.2.10 static int rpm_vreg_get_voltage(struct regulator_dev *rdev)

Gets the voltage on a regulator. The regulator must be enabled.

Parameters

in	<i>rdev</i>	Voltage/current regulator class device; one for each regulator.
----	-------------	---

Returns

On success, returns the voltage of the specified regulator; otherwise, it returns 1 (unknown voltage).

Dependencies

Call `msm_rpm_dev_probe` before calling this function.

3.2.11 static int rpm_clk_prepare(struct clk *clk)

Prepares the clock before setting the clock rate to the Active or Sleep state.

Parameters

in	<i>clk</i>	Global structure for clock parameters.
----	------------	--

Returns

If the clock sets the Active state, it returns TRUE; otherwise, it returns FALSE.

Dependencies

Call `msm_rpm_dev_probe` before calling this function.

`clk_prepare` must be called before `clk_enable`.

3.2.12 static void rpm_clk_unprepare(struct clk *clk)

Unprepares the clock rate that was set in the Active or Sleep state.

Parameters

in	clk	Global structure for clock parameters.
----	-----	--

Returns

None.

Dependencies

Call `msm_rpm_dev_probe` before calling this function.

`clk_disable` must be called before `clk_unprepare`.

3.2.13 static int rpm_clk_set_rate(struct clk *clk, unsigned long rate)

Used to specify a new rate for a clock.

Parameters

in	clk	Global structure for clock parameters.
in	long rate	Clock rate in Hz.

Returns

If the clock rate sets properly for a specified clock message ID, the API returns TRUE; otherwise, it returns an error message.

Dependencies

Call `msm_rpm_dev_probe` before calling this function.

3.2.14 static unsigned long rpm_clk_get_rate(struct clk *clk)

Gets the clock rate of a specified subsystem clock.

Parameters

in	clk	Global structure for clock parameters.
----	-----	--

Returns

Returns the clock frequency of the specified module.

Dependencies

Call `msm_rpm_dev_probe` before calling this function.

A Source Code References

API source code files:

- /kernel/drivers/regulator/rpm-smd-regulator.c
- /kernel/drivers/clk/qcom/clock-rpm.c
- /kernel/include/soc/qcom/rpm-smd.h
- /kernel/include/linux/regulator/driver.h
- /kernel/arch/arm/mach-msm/include/mach/rpm-regulator.h

Errors are defined in the following header file:

- /kernel/include/uapi/asm-generic/errno-base.h

B References

B.1 Acronyms and terms

Acronym or term	Definition
DDR	Double data rate
KVP	Key value pair
MPM	MSM power manager
NPA	Node power architecture
PMIC	Power management integrated circuit
RPM	Resourc power manager
SMD	Shared memory driver
SPMI	System power management interface
WDOG	Watchdog